UAV Intelligence Engine — Phase-Wise Documentation (Weeks 1–2)

Offline, air-gapped, multi-modal UAV mission analysis pipeline. This document covers everything completed and verified through Week 2, in build order, with the reasoning behind each decision — not just what was done, but why.

Week 1 — Environment Setup

Goal

Get a fully offline-installable Python environment running on the facility's air-gapped Windows workstation, using a wheelhouse (pre-downloaded packages) built at home and transferred physically.

What was built

- `requirements.txt` — pinned package list, CPU-only (no CUDA toolkit on the facility PC).
- `scripts/build_wheelhouse.bat` — downloads every dependency at home into `wheelhouse/`, computes checksums.
- `scripts/install.bat` — 13-step offline installer: integrity check → Python check → venv creation → wheelhouse install → FFmpeg check → model-weight checks → config smoke test.
- `config/paths.yaml`, `config/ontology.yaml`, `src/config.py` — central configuration (mission phases, event types, scoring weights, file paths).

Problems hit, and how they were resolved

1. **Wrong Python version at the facility.** The original plan targeted Python 3.10. The facility PC's `python` command resolves to 3.14 by default (multiple versions are installed there). `install.bat` originally called plain `python`, so it silently built the venv with the wrong interpreter, and every wheel install failed with "no matching distribution." **Resolution:** rather than continue fighting Windows PATH resolution across repeat visits, the whole project was retargeted at **Python 3.14** — rebuilding the entire wheelhouse for `cp314`, and changing `install.bat`/`build_wheelhouse.bat` to call `py -3.14` explicitly instead of relying on PATH order.
2. **Silent NumPy/PyTorch ABI mismatch.** An unpinned `numpy` dependency resolved to a 2.x version while PyTorch expected NumPy 1.x's C-API. This produced a `UserWarning` at import time, not a hard failure — meaning `INSTALL OK` could print while a real, silent compatibility problem sat underneath. **Resolution:** pinned `numpy` explicitly and verified a clean install log with no `_ARRAY_API` warning text.

3. **Missing `lap` dependency for tracking.** Discovered only when actually running YOLO tracking (not caught by earlier import checks) — ByteTrack requires the `lap` package, which `ultralytics` auto-installs online but which was absent from the offline wheelhouse. Would have hard-failed at the facility with no way to fix it. **Resolution:** added `lap>=0.5.12` to `requirements.txt`, rebuilt that wheel into the wheelhouse.
4. `pytest` **unavailable offline.** Testing tools were deliberately kept out of the shipped wheelhouse initially; this meant no way to verify the install at the facility without internet. **Resolution:** added `pytest>=9.0` under a clearly-marked "dev/testing only" section of `requirements.txt` so it installs automatically via `install.bat`, with no separate internet-dependent step needed on-site.

Deliverables

- `src/media/probe.py` — wraps the bundled `ffprobe.exe` (never system PATH) to report duration, resolution, fps, codec, and audio presence.
- `src/media/sample.py` — wraps the bundled `ffmpeg.exe` to extract frames at a configured rate (`sampling.detection_fps`, currently 3 fps), letterboxed to the YOLO inference size (640px).
- 6 automated tests (`tests/test_media.py`), self-contained — they generate their own short synthetic clips via ffmpeg rather than depending on real footage.

Verified on real data

`Sample.mp4` (3840×2160, HEVC, 17.55s, AAC audio) → correctly reported all metadata, correctly extracted 53 frames (17.55s × 3fps, rounded up).

Tag: `v0.1.0` (environment) → `v0.2.0` (media ingestion)

Week 2 — Vision: Detection, Tracking, Keyframes

Goal

Detect and track objects across the sampled frames, and select one representative frame per tracked object for later use in captioning and reporting.

What was built

`src/vision/detect.py` — runs YOLO11n over every sampled frame using ultralytics' built-in ByteTrack tracker (`persist=True` across the frame sequence, so track identity carries forward frame to frame instead of resetting each call). Outputs:

- `detections.json` — every individual detection, every frame (raw signal).
- `tracks.json` — one row per tracked object: class, first/last frame seen, how many frames it was actually detected in, mean confidence.

`src/vision/keyframes.py` — for each track, scans all of its recorded detections and selects the single one with the highest confidence score, copies that frame into `video/keyframes/`, and writes a manifest (`keyframes.json`) linking track → keyframe file → confidence → bounding box. This deliberately picks the *best* frame, not the first — verified by a dedicated test where the correct answer is a track's middle frame, not its first or last.

How detection and tracking actually work (mechanism)

1. YOLO11n (pre-trained on COCO, 80 general object classes — not UAV-specific) outputs boxes + confidence per frame.
2. Anything below `detection.confidence_threshold` (0.25) is discarded.
3. ByteTrack predicts each existing track's expected position using a Kalman filter (motion-based extrapolation), then matches new detections to tracks in two passes — high-confidence boxes first, then a second pass that tries to recover low-confidence boxes against still-unmatched tracks (its core design advantage over simpler trackers, which discard low-confidence boxes as noise).
4. Each track has a lifecycle: new → tracked → lost (briefly unmatched, still held) → removed (unmatched too long). This is what allows a track to survive a one-frame gap without being treated as a new object.

Problems hit, and how they were resolved

- **Missing `lap` dependency** — see Week 1, discovered during this phase's testing, fixed the same way.
- **File-transfer reliability** — several code files arrived as empty 0-byte stubs on the facility-bound machine after download, traced back to browser download/move issues rather than any code problem. Resolved by writing file contents directly via PowerShell here-strings instead of relying on file download + move.

Verified on real data

`Sample.mp4` → 53 frames → **~117–128 detections across ~16–17 tracks** (small run-to-run variance is expected and normal — YOLO/ByteTrack involve non-deterministic tie-breaking; the pattern, not the exact count, is what matters). Object classes found: person, motorcycle, car. Confidence range roughly 0.40–0.94. Keyframe selection correctly picked each track's true best-confidence frame in every run.

Tests

10 automated tests (`tests/test_vision.py` + `tests/test_keyframes.py`), covering: correct class detection, track continuity across frames, output files actually written, missing-input error handling, and — the meaningful one — correct "best, not first" keyframe selection using a fixture where the right answer is deliberately not the first frame.

Tag: `v0.3.0` (detection + tracking) → `v0.3.1` (keyframe selection)

Current overall status

Metric	Value
Weeks complete	1 and 2, of 6
Automated tests passing	19 / 19
Verified against	Real UAV footage (`Sample.mp4`), not just synthetic clips
Offline install	Fully reproducible via `install.bat`, no internet needed at the facility
Model weights staged	YOLO11n, Faster-Whisper (small, int8) — downloaded, not yet integrated into code
Not yet started	Weeks 3–6: event scoring, highlight generation, audio transcription, LLM reasoning, TTS, PDF report, dashboard

Design decisions locked in for upcoming weeks (not yet built)

- Output is conditional: a highlight reel is always produced; a spoken TTS summary, PDF report, and dashboard narration are only produced when the input actually has an audio track (`probe.py`'s `has_audio` field already available for this).
- The local LLM (Qwen3, via Ollama) is never shown raw video. It receives a structured JSON evidence package assembled from detections, scored events, per-keyframe captions (from a vision-language model, run separately per still image), and transcript segments.
- Highlight clips include a 2–3 second lead-in before each scored event, not just the exact detected frame — an editing decision made at highlight assembly time, not a change to the detection model.
- Text-to-speech supports English and Hindi, selected automatically from Faster-Whisper's own detected transcript language (no separate language-ID step needed).
- The final highlight reel is bookended with takeoff footage at the start and landing footage at the end when present, regardless of their individual event scores — open question: how takeoff/landing are actually identified in footage with no mission telemetry available.